

Especificação do Projeto

Kevin Rodrigues Nunes
15676030
T94

Victor Yodono
13829040
T94

Isabelle da Silva Tobias
15525991
T94

Kauã Lima de Souza
15674702
T94

Resumo — Este documento apresenta a especificação do projeto desenvolvido pelo grupo na disciplina ACH2147 - Desenvolvimento de Sistemas de Informação Distribuídos. O objetivo é registrar as decisões de projeto e detalhá-las o suficiente para que qualquer desenvolvedor com conhecimento na área seja capaz de implementá-las. Para isso, o relatório apresenta uma visão geral do sistema, as decisões arquiteturais tomadas e os protocolos adotados para resolver problemas clássicos de sistemas distribuídos.

I. INTRODUÇÃO

Este documento apresenta a especificação arquitetural para um projeto de coordenação distribuída de robôs de transporte. Esses robôs são conhecidos como AGVs (*Automated Guided Vehicles* ou Veículos Guiados Automatizados) e são amplamente utilizados em armazéns logísticos para movimentar cargas.

Na grande maioria das indústrias, os AGVs são controlados por um servidor centralizado que calcula as rotas, distribui as tarefas e evita colisões. No entanto, para este projeto acadêmico, optamos por uma abordagem **descentralizada**, a fim de aplicar na prática os conceitos teóricos estudados no livro *Sistemas Distribuídos: Princípios e Paradigmas* (Tanenbaum) [1], que é parte da bibliografia do curso. Ao remover o servidor central, eliminamos o Ponto Único de Falha (SPOF) e transferimos a inteligência para os próprios robôs, que precisam usar protocolos de comunicação, eleição e consenso para operarem de forma autônoma e cooperativa no mesmo ambiente.

II. VISÃO GERAL DO PROJETO

O projeto consiste em desenvolver um software que simula uma frota de AGVs. O foco central da solução é permitir que os robôs (que funcionam como nós da rede) decidam entre si quem vai executar qual tarefa de transporte e mantenham seus estados sincronizados, tudo isso sem depender de um orquestrador central.

III. REQUISITOS

Esta seção define o escopo do sistema através de seus requisitos. Os Requisitos Funcionais descrevem o que o sistema deve fazer, enquanto os Não-Funcionais estabelecem as restrições de qualidade (como confiabilidade e estrutura da rede).

A. Requisitos Funcionais

Os requisitos a seguir especificam o comportamento operacional de cada AGV:

- Descoberta Dinâmica de Peers:** Os AGVs devem descobrir automaticamente o IP e a porta de outros AGVs ativos na rede através de mensagens multicast.
- Distribuição de Tarefas:** Qualquer AGV ativo deve ser capaz de receber novos pedidos (enviados por um cliente externo) e repassar a informação para a frota, via broadcast.
- Proposta e Sincronia de Lotes:** O líder atual da frota deve agrupar tarefas pendentes em um lote (**Batch**) e enviar para todos, aguardando confirmações (**BATCH_ACK**) para manter todos na mesma página.
- Alocação Determinística (Leilão Local):** Cada AGV deve calcular localmente qual robô está mais perto da tarefa (usando Distância de Manhattan). Esse cálculo deve usar o *snapshot* (foto do estado das posições) enviado pelo líder junto com a proposta do lote, garantindo que todos os robôs usem os mesmos dados para decidir o vencedor de forma síncrona.
- Simulação de Deslocamento:** Os AGVs devem simular sua movimentação pelo grid com base em rotas calculadas pelo algoritmo A*, atualizando suas posições a cada 300ms.
- Recuperação de Tarefas Órfãs:** O líder deve perceber se um AGV com tarefa em andamento cair,

pegando essa tarefa de volta e colocando na fila para ser feita por outro robô.

7. **Eleição Ativa (Bully):** Se o líder cair (parar de mandar *heartbeats*), o primeiro nó que perceber deve iniciar uma eleição usando o Algoritmo de Bully para escolher um novo líder.

B. Requisitos Não-Funcionais

1. **Confiabilidade de Rede:** Como usaremos UDP, a perda de pacotes deve ser tratada no nível da aplicação usando o **Protocolo SRM** (Scalable Reliable Multicast), que usa números de sequência e pedidos de retransmissão via NACK.
2. **Arquitetura Descentralizada (P2P):** O sistema não pode ter banco de dados central nem orquestrador. Todos os nós operam como pares.
3. **Consistência Sequencial:** O histórico de tarefas e do mapa deve ser idêntico em todos os robôs ativos.

IV. ARQUITETURA

A arquitetura foi desenhada para garantir a autonomia dos nós. Optou-se por separar bem as regras de negócio da parte de comunicação e rede.

A. Modelo de Arquitetura Distribuída

O sistema adota uma topologia **Peer-to-Peer (P2P) Descentralizada**. Não há servidor de controle. Todos os AGVs possuem o mesmo código e cooperam de forma igualitária. Para propósitos de simulação, especificaremos um "Gerador de Pedidos": processo que atua apenas como um cliente externo que injeta tarefas na rede, mas não gerencia os robôs nem guarda dados de estado. Ainda, teremos um módulo de visualização que simulará a posição e movimentação dos AGVs no armazém, recebendo as atualizações periódicas de status e localização e exibindo em um grid virtual.

B. Middleware Customizado

Para fins de aprendizado, não utilizaremos ferramentas prontas como RabbitMQ ou JGroups. A comunicação será feita via sockets UDP e a própria aplicação gerenciará a confiabilidade e a ordem das mensagens.

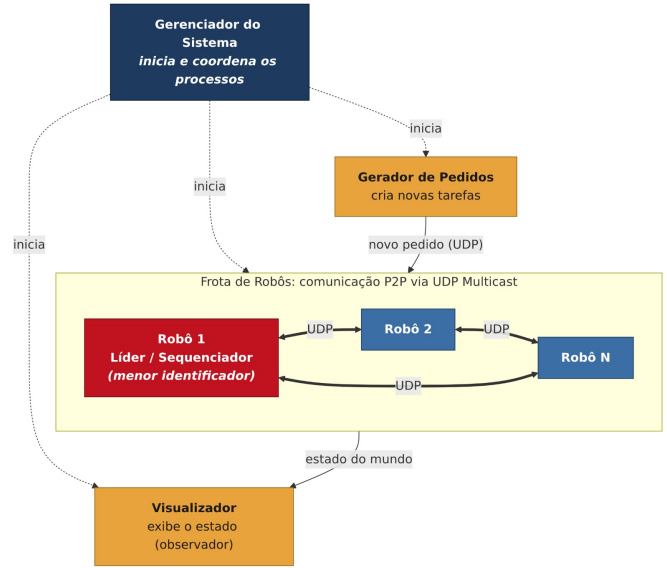


FIGURE I: COMPONENTES DO SISTEMA

V. PROCESSOS

O gerenciamento de processos define como o sistema lidará com os estados, concorrência e isolamento.

A. Servidor stateful ou stateless?

O Gerador de Pedidos é **Stateless** (não guarda o estado da frota). Já os nós AGVs são altamente **Stateful**, mantendo na memória a própria posição, a rota, o relógio lógico e a lista de outros robôs conhecidos.

B. Uso de Threads

Cada AGV trabalha com múltiplas threads operando ao mesmo tempo:

1. **Thread de Movimento:** Atualiza a posição no grid a cada 300ms.
2. **Thread de Rede:** Fica escutando a porta UDP para receber mensagens dos outros nós.
3. **Thread de Heartbeat:** Envia avisos de que o robô está vivo a cada 1 segundo e checka se algum par parou de responder.

C. Técnicas de Virtualização

Por se tratar de uma simulação, encontra-se a necessidade de resolver a **heterogeneidade do hardware** (virtualização a nível de processo/linguagem) dos computadores de teste da equipe, garantindo que o comportamento de concorrência e rede P2P seja exatamente idêntico em qualquer nó de execução, independente do SO nativo (Windows/Linux/macOS). Além disso, o uso de containers é uma técnica adicional que garante que falhas críticas de recursos em um AGV (ex: estouro de memória, vazamento de recursos) não afetem o sistema como um todo.

mento de sockets ou loops infinitos de thread) fiquem contidos no seu próprio container, impedindo que degradem a máquina física hospedeira ou afetem o funcionamento dos outros AGVs virtuais que compartilham o mesmo hardware de simulação.

VI. COMUNICAÇÃO

O sistema foi desenvolvido para operar sob protocolos eficientes, porém com lógicas extras para lidar com perdas de pacotes na rede.

A. Tipo de Comunicação

A base física utiliza **UDP Multicast**. Para garantir que os dados importantes não se percam, construímos uma camada confiável baseada no SRM. Cada pacote tem um número de sequência. Assim, se um robô percebe que pulou um número, ele envia um **NACK** pedindo para retransmitirem aquela mensagem.

B. Tipos de Mensagens

O sistema trafega mensagens como:

1. **HEARTBEAT**: Mensagem periódica avisando que o robô está ativo.
2. **ELECTION, OK, COORDINATOR**: Mensagens exclusivas do algoritmo de eleição Bully.
3. **NEW_ORDER**: Novo pedido injetado pelo sistema externo.
4. **BATCH_PROPOSAL** e **BATCH_ACK**: Proposta de lote de tarefas feita pelo líder e a confirmação de recebimento pelos outros nós.
5. **NACK_REQUEST** e **NACK_RESPONSE**: Mensagens do protocolo SRM para pedir pacotes perdidos.

C. Formato das Mensagens (Payload)

Para garantir a padronização, todas as mensagens são formatadas em JSON. O cabeçalho obrigatório de todo pacote contém:

- **senderId**: Identificador estático do emissor (ex: agv-01).
- **sequenceNumber**: Número inteiro para o controle do protocolo SRM.
- **lamportTimestamp**: Inteiro com o valor atual do relógio lógico do nó.
- **messageType**: O tipo da ação (ex: BATCH_PROPOSAL).
- **payload**: Objeto com os dados específicos (como as coordenadas de uma nova tarefa).

D. Fluxo Lógico (Leilão de Lote)

O fluxo para aceitar e distribuir uma tarefa ocorre da seguinte maneira:

1. O **Gerador** envia um **NEW_ORDER** em Multicast.
2. O **Líder** agrupa os pedidos pendentes em um lote e envia o **BATCH_PROPOSAL** (junto com a posição de todos os robôs) em Multicast.
3. Os **AGVs** recebem a proposta, atualizam seus relógios e respondem com **BATCH_ACK** em Multicast.
4. Ao verem que todos os robôs ativos enviaram o **ACK**, cada AGV roda o cálculo de menor distância localmente.
5. O robô vencedor se movimenta, enquanto os outros marcam aquela tarefa como "em andamento" em suas memórias.

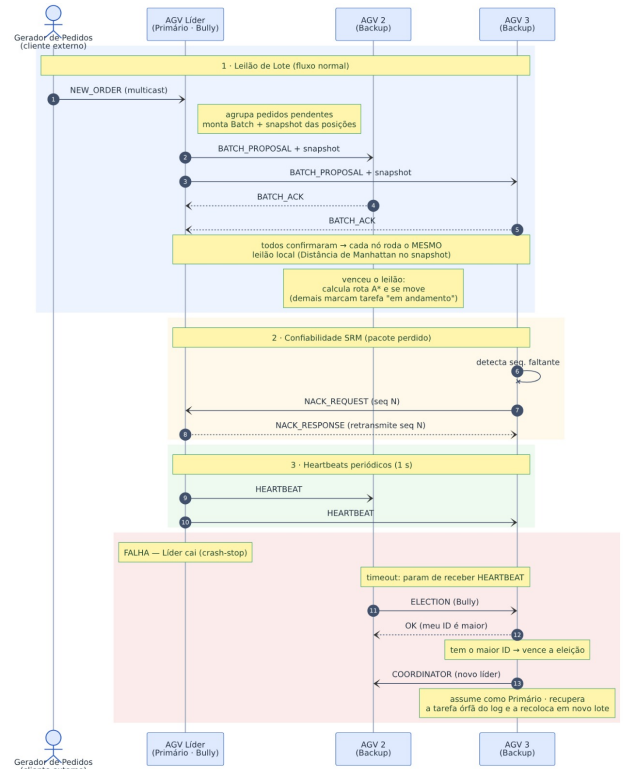


FIGURE II: DIAGRAMA DE SEQUÊNCIA DE MENSAGENS

VII. NOMEAÇÃO

Para que um nó encontre o outro em uma rede P2P sem um servidor DNS, adotamos estratégias descentralizadas.

A. Garantia de Unicidade dos Robôs

Cada robô recebe um nome estático no arquivo de configuração (ex: **agv-01**). Esse nome é juntado com um código gerado aleatoriamente (UUID) quando o sistema liga. Isso garante que, se o robô reiniciar, os outros saibam que é uma nova sessão daquele mesmo robô.

B. Resolução de Nomes

Adotamos o **Flat Naming Descentralizado**. Cada AGV tem uma tabela local na memória. Quando ele recebe um pacote de HEARTBEAT, ele lê o IP e a porta de quem mandou e salva nessa tabela.

C. Identificação de Pedidos e Lotes

Outros recursos do sistema também precisam de IDs únicos:

- **Pedidos (Orders):** O Gerador externo cria um ID único usando um UUID.
- **Lotes (Batches):** O Líder gera o ID do lote juntando o nome dele com o valor do seu relógio lógico no momento (ex: `agv-01-L42`), garantindo que os IDs nunca se repitam no tempo e no espaço.

VIII. COORDENAÇÃO

Sistemas distribuídos precisam de regras matemáticas para definir a ordem dos eventos e quem manda na rede.

A. Relógio Lógico e Ordenação Total

O sistema usa os **Relógios Lógicos de Lamport** para ordenar os eventos. Como pode acontecer de dois eventos ganharem o mesmo número no relógio de Lamport, precisamos de um critério de desempate para gerar uma *Ordenação Total*. Nesse caso, usamos o nome estático do robô: se houver empate no número do relógio, o evento do robô com o menor nome (ex: `agv-01` ganha de `agv-02`) é considerado o mais antigo.

B. Exclusão Mútua Distribuída

A prevenção de colisões via exclusão mútua clássica não será implementada agora. A ideia proposta é uma Reserva Dinâmica por Rota: um robô só pedirá exclusão mútua se o caminho que ele calculou bater com o caminho que outro robô está fazendo no momento.

C. Algoritmo de Eleição (Bully)

O líder da rede é escolhido usando o **Algoritmo de Bully (Valentão)**. Se um robô percebe que o líder parou de mandar heartbeats, ele manda uma mensagem de eleição. A regra é simples: o robô ativo com o maior ID estático configurado (ex: `agv-03` manda mais que `agv-01`) ganha a eleição e vira o novo líder. O UUID dinâmico gerado ao ligar o robô é ignorado nessa decisão.

IX. CONSISTÊNCIA E REPLICAÇÃO

Para evitar que dois robôs tentem buscar a mesma mercadoria, todos precisam enxergar a mesma lista de tarefas

pendentes.

A. Replicação de Máquina de Estado

Usamos a Replicação de Máquina de Estado para garantir **Consistência Sequencial**. Isso significa que todos os robôs processam as tarefas exatamente na mesma ordem, gerando o mesmo estado final no mapa e no inventário de cada um.

B. Protocolo de Consistência

Aproveitamos um modelo **Primary-Backup Dinâmico**. O líder eleito pelo Bully atua como Primário (montando os lotes), enquanto os outros atuam apenas como Backups, recebendo e validando os lotes.

X. TOLERÂNCIA A FALHAS

O sistema deve sobreviver a problemas de comunicação e quedas de processos.

A. Falhas Toleradas

- **Omissão de Canal:** Pacotes UDP perdidos são resolvidos com o protocolo SRM (NACKs e retransmissões).
- **Crash-Stop (Queda de nó):** Se o robô encarregado de uma entrega travar, os outros vão notar que ele parou de mandar HEARTBEATS. O líder vai pegar a tarefa não finalizada no log e colocar em um novo lote para outro robô fazer.

B. Confiabilidade ou Disponibilidade?

O sistema favorece a Confiabilidade e a Segurança (*Safety*). Se a rede cair e os robôs não conseguirem trocar mensagens de ACK, eles ativam o modo Fail-Safe e simplesmente param de se mover, evitando colisões.

XI. SEGURANÇA

Em robótica, a segurança tem duas frentes: *Safety* (Física) e *Security* (Dados).

A. Safety (Segurança Operacional e Física)

Garantimos a integridade do galpão adotando o Fail-Safe. O robô para imediatamente se perder a conexão com a rede. Além disso, o determinismo do algoritmo de lote garante que a mesma coordenada do chão nunca será reservada para dois robôs ao mesmo tempo.

B. *Security (Segurança Cibernética)*

Assumimos que a responsabilidade de segurança será encargo da infraestrutura do armazém (externo ao projeto). Como os robôs operam no Wi-Fi industrial privado local, delegamos a segurança contra invasões para firewalls, VLANs e outras técnicas. Isso poupa os robôs de gastarem processamento com criptografia pesada de mensagens, deixando a CPU livre para calcular as rotas em tempo real.

XII. DECISÕES DE PROJETO E TESTES

Para validar o sistema distribuído e garantir que ele resista a problemas reais, adotamos a seguinte estratégia de testes:

- **Simulação Concorrente:** Rodar vários processos ao mesmo tempo interagindo no mesmo mapa lógico.
- **Injeção de Falhas:** Ferramentas para atrasar ou derrubar pacotes UDP de propósito durante a simulação.

- **Simulação de Queda:** Forçar o desligamento abrupto (`kill -9`) de um robô no meio do caminho para testar se o líder recupera a tarefa corretamente.

XIII. CONCLUSÃO

Esta especificação consolida as bases para o desenvolvimento do sistema de frota de AGVs, alinhando as exigências práticas de um sistema logístico com a teoria clássica de sistemas distribuídos. Ao escolher não usar um servidor central e delegar a tomada de decisão para os nós pares. Utilizamos principalmente algoritmos como Bully, Lamport e replicação de máquina de estado, propondo um modelo robusto, tolerante a falhas e ideal para consolidar os conceitos da disciplina.

REFERÊNCIAS

- [1] TANENBAUM, Andrew S.; VAN STEEN, Maarten. *Sistemas Distribuídos: Princípios e Paradigmas*. 2. ed. São Paulo: Pearson Prentice Hall, 2007.